

Segurança em Computação

INE5429-07208

Thaís Bardini Idalino

Integridade e autenticação

Sumário

- Definições básicas
- Funções de hash criptográficas
- Códigos de autenticação de mensagens
- Atividades práticas

Tipos de Ataques

1. Ataques à Confidencialidade

- a. Divulgação: Liberação do conteúdo da mensagem a terceiros não autorizados.
- b. Análise de Tráfego: Descoberta de padrões de comunicação e metadados.

2. Ataques à Autenticidade e Integridade

- a. Máscara: Inserção de mensagens falsas, fingindo ser uma entidade autorizada.
- b. Modificação de Conteúdo: Alteração de dados, como inserção, exclusão ou reordenação.
- c. Modificação de Sequência: Mudança na ordem das mensagens.
- d. Modificação de Tempo: Atraso ou repetição de mensagens válidas.

3. Ataques de Não-Repúdio

- a. Não Reconhecimento na Origem: O emissor nega ter enviado uma mensagem.
- b. Não Reconhecimento no Destino: O receptor nega ter recebido uma mensagem.

Soluções Criptográficas para Cada Ataque

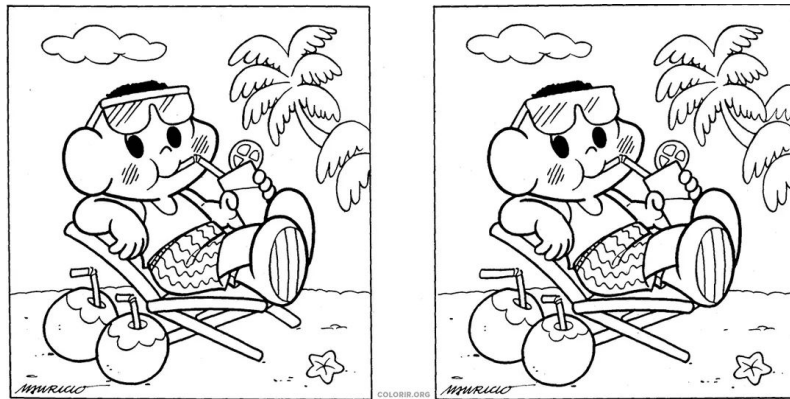
1. **Confidencialidade:** Garantida através de mecanismos de cifragem de mensagem.
2. **Autenticação e Integridade:** Combatidas com autenticação de mensagem (e.g., MACs, hashes criptográficos).
3. **Não-Repúdio:** Exige o uso de assinaturas digitais, que também podem combater os ataques de autenticação e integridade.

Na aula de hoje..

- Vamos focar na garantia de integridade e autenticação de mensagem
- **Integridade:** garantia de que o conteúdo de uma mensagem não foi alterado por terceiros não autorizados desde o momento do envio.
 - funções de hash criptográficas
- **Autenticação:** mecanismo utilizado para verificar se as mensagens recebidas vêm da origem afirmada e não foram alteradas (integridade).
 - códigos de autenticação de mensagens e assinaturas digitais

Solução para integridade

- Se eu tenho o dado original salvo em um local seguro, posso usar ele para comparação e identificação de modificações
- **Problema:** se o dado é muito grande, preciso de muito espaço de armazenamento
- **Solução:** funções de hash



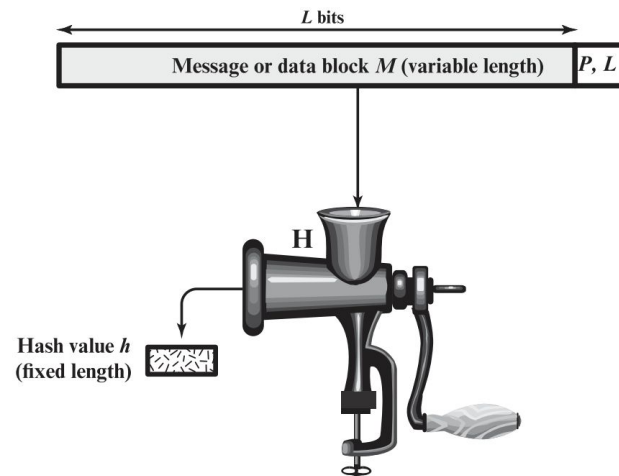
Fonte: <https://tinyurl.com/7-erros>

Sumário

- Definições básicas
- **Funções de hash criptográficas**
 - Definições básicas
 - Requisitos e segurança
 - O algoritmo SHA
- Códigos de autenticação de mensagens
- Atividades práticas

Funções de hash

- Uma *função de hash* recebe como entrada uma mensagem M de tamanho variável e produz uma saída $H(M)$ de tamanho fixo
 - chamamos a saída $h = H(M)$ de *hash*
- São conhecidas como funções de **caminho único**.
- As saídas devem ser bem distribuídas e com aparência aleatória.
- Uma mudança em qualquer bit de uma mensagem M deve resultar em um valor $H(M)$ diferente.
 - conhecido como efeito avalanche



P, L = padding plus length field

Figure 11.1 Cryptographic Hash Function; $h = H(M)$

Imagens: W. Stallings. *Cryptography and network security*. Cap 11.1

Funções de hash **criptográficas**

- Uma *função de hash* **criptográfica** é utilizada para prover garantia da integridade de dados
 - Possui certos requisitos de segurança
- Ela é utilizada para construir um "resumo" do dado
 - Se o dado foi alterado, o seu resumo se torna inválido

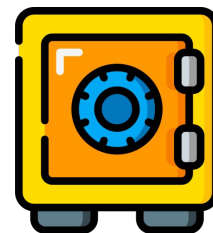
Exemplo

- Para verificar a integridade de um dado:
 - calculamos seu resumo/*hash* e armazenamos em um local seguro
 - o dado pode ser armazenado ou transmitido através de um canal inseguro
 - a verificação posterior consiste em recalcular o resumo/*hash* e verificar se o valor é igual ao armazenado

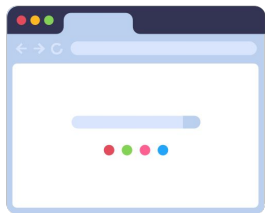


Função de
hash

56b497f1a1929b



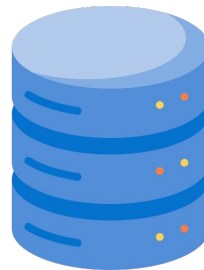
Exemplo



minhasenha123

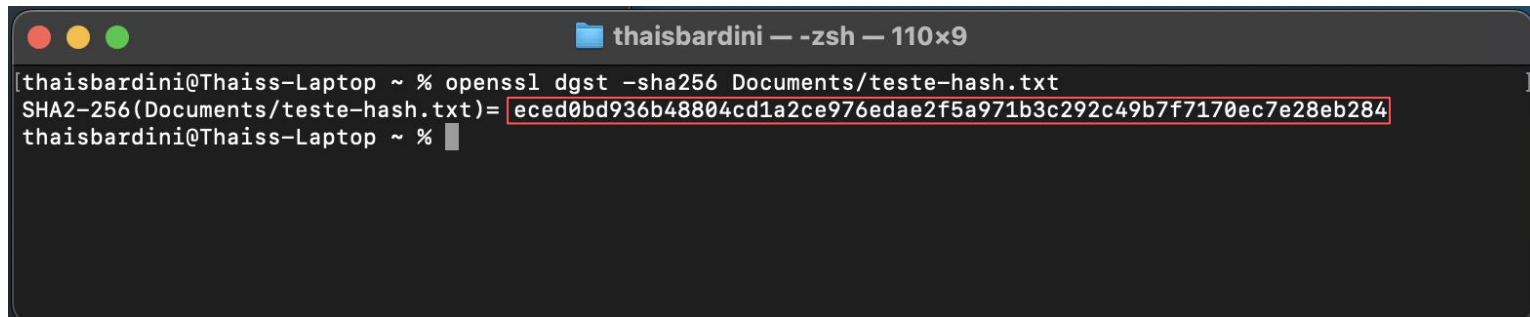
Função de
hash

56b497f1a1929b



Exemplo

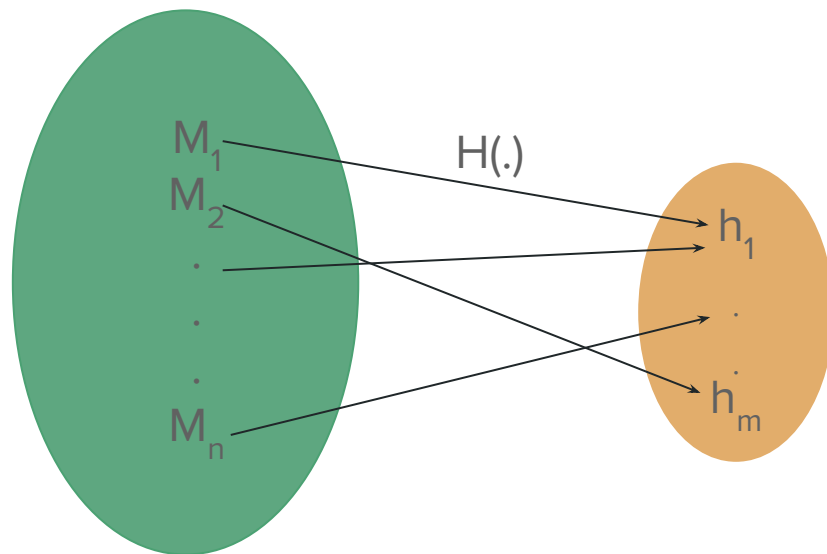
openssl dgst -sha256 nome-do-arquivo



```
thaisbardini — -zsh — 110x9
[thaisbardini@Thaiss-Laptop ~ % openssl dgst -sha256 Documents/teste-hash.txt
SHA2-256(Documents/teste-hash.txt)= eced0bd936b48804cd1a2ce976edae2f5a971b3c292c49b7f7170ec7e28eb284
thaisbardini@Thaiss-Laptop ~ % ]
```

Colisão

- Como a entrada tem tamanho arbitrário e a saída tem tamanho fixo, valores diferentes de M resultam no mesmo hash h .
- Chamamos isso de *colisão de hash*.



Colisão



```
thaisbardini — -zsh — 80x24
[thaisbardini@Thaiss-Laptop ~ % openssl dgst -md5 Documents/ship.jpg
MD5(Documents/ship.jpg)= 253dd04e87492e4fc3471de5e776bc3d
[thaisbardini@Thaiss-Laptop ~ % openssl dgst -md5 Documents/plane.jpg
MD5(Documents/plane.jpg)= 253dd04e87492e4fc3471de5e776bc3d
thaisbardini@Thaiss-Laptop ~ % █
```

Requisitos

Requisitos de segurança de uma função de hash **criptográfica**:

- Tamanho de entrada variável e de saída fixo
- Fácil de computar para qualquer entrada
- **Resistência à pré-imagem**: dado $H(x)$, é inviável achar x
- **Resistência à 2a pré-imagem**: dados x e $H(x)$, é inviável encontrar $y \neq x$ com $H(x) = H(y)$
- **Resistência à colisão**: inviável achar dois valores $x \neq y$ com $H(x) = H(y)$
- Saída passa nos testes de pseudo-aleatoriedade

Segurança das funções de hash

- **Ataque de força bruta:** depende apenas do tamanho da saída da função, e não do algoritmo.
- Assuma uma função H com saída de m bits.
- Para encontrar x dado $h = H(x)$, o esforço é proporcional a 2^m tentativas
 - estatisticamente, encontramos x após tentar metade das possibilidades, ou seja, 2^{m-1}
 - primeira e segunda pré-imagens
- Para encontrar dois valores diferentes x e y tal que $H(x) = H(y)$, precisamos de um esforço muito menor, graças ao **paradoxo do aniversário**.
 - de apenas $2^{m/2}$ tentativas
 - colisão

Paradoxo do aniversário

- Probabilidade de colisão pode ser avaliada usando o **paradoxo do aniversário**
 - Em um grupo de 23 pessoas aleatoriamente selecionadas, duas delas têm a mesma data de aniversário com probabilidade de pelo menos 50%
 - 365 dias do ano, $\sqrt{365} = 22.3$
 - Encontrar duas pessoas com o mesmo aniversário é como achar uma colisão no hash
- Segurança da função SHA256:
 - Espaço de 2^{256} possibilidades de hash
 - Paradoxo: $\sqrt{2^{256}}$ para 50% de chances
 - $2^{256/2} = 3.4028237e+38$ tentativas

As { the } Dean of Blakewell College, I have { had the pleasure of knowing } Cherise
 Rosetti for the { last } four years. She { has been } { a tremendous } { asset to }
 { our } school. I { would like to take this opportunity to } recommend Cherise for your
 { the } graduate program. I { am } { confident } { that } { she } will
 { continue to } succeed in her studies. { She } is a dedicated student and
 { thus far her grades } { have been } { exemplary } . In class,
 { her grades thus far } { are } { excellent } .
 { she } { has proven to be } a take-charge { person } { who is } able to
 { Cherise } { has been } { individual } { — } successfully develop plans and implement them.

{ She } has also assisted { us } in our admissions office. { She } has
 { Cherise } successfully demonstrated leadership ability by counseling new and prospective students.

{ Her } advice has been { a great } help to these students, many of whom
 { Cherise's } { of considerable }

have { taken time to share } their comments with me regarding her pleasant and
 { shared }

{ encouraging } attitude. { For these reasons } I
 { reassuring } { It is for these reasons that }

{ highly recommend } Cherise { without reservation } . Her { ambition } and
 { offer high recommendations for } { unreservedly } { drive }

{ abilities } will { truly } be an { asset to } your { establishment } .
 { potential } { surely } { plus for } { school }

Figure 11.7 A Letter in 2^{38} Variations

Imagens: W. Stallings. *Cryptography and network security*. Cap 11.3

Funções de hash

- As funções mais utilizadas hoje em dia são as da família Secure Hash Algorithm (SHA)
- Desenvolvidas pelo Instituto Nacional de Padrões e Tecnologia (NIST) e padronizadas em 1993
 - Em 1995, a versão SHA-0 foi comprometida e substituída pela nova versão SHA-1
 - Em 2002, três novas versões foram criadas e chamadas de SHA-2: SHA-256, SHA-384 e SHA-512
 - Em 2008, foi incluída a versão SHA-224
 - Em 2015, foram adicionadas a SHA-512/224 e SHA-512/256
 - Foi encontrado um ataque de colisão no SHA-1 e hoje ele não é mais recomendado.
 - Uma competição do NIST anunciada em 2007 produziu a nova geração de funções de hash, a SHA-3. Ela foi padronizada em 2012.
- A especificação dos algoritmos é apresentada atualmente na [FIPS PUB 180-4](#), conhecida como Secure Hash Standard (SHS)

Table 11.3 Comparison of SHA Parameters

Algorithm	Message Size	Block Size	Word Size	Message Digest Size
SHA-1	$< 2^{64}$	512	32	160
SHA-224	$< 2^{64}$	512	32	224
SHA-256	$< 2^{64}$	512	32	256
SHA-384	$< 2^{128}$	1024	64	384
SHA-512	$< 2^{128}$	1024	64	512
SHA-512/224	$< 2^{128}$	1024	64	224
SHA-512/256	$< 2^{128}$	1024	64	256

Note: All sizes are measured in bits.

Imagens: W. Stallings. *Cryptography and network security*. Cap 11.5

SHA-512

- **Entrada:** mensagem $< 2^{128}$ bits
- Entrada processada em blocos de 1024 bits cada
- **Saída:** resumo de 512 bits

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

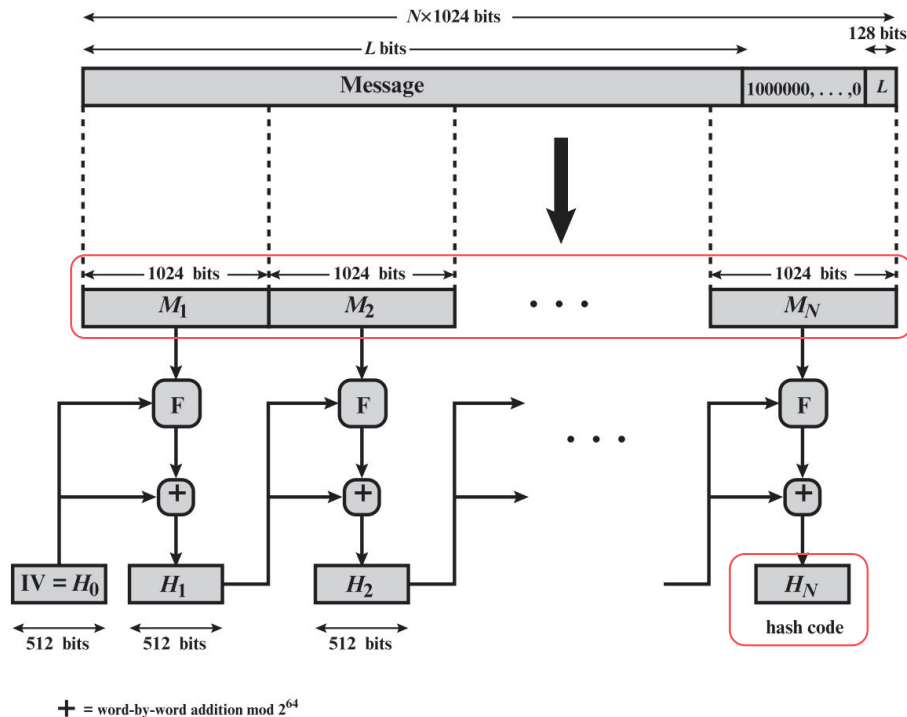


Figure 11.9 Message Digest Generation Using SHA-512

SHA-512

- **Entrada:** mensagem $< 2^{128}$ bits
- Entrada processada em blocos de 1024 bits cada
- **Saída:** resumo de 512 bits
- Padding 10...0 é adicionado para garantir comprimento específico

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

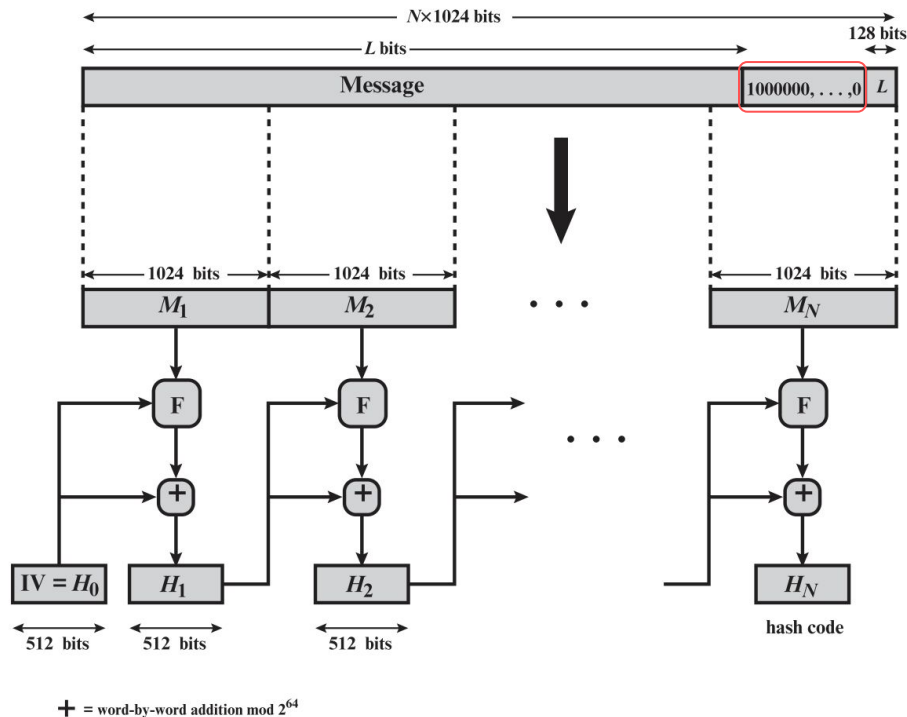


Figure 11.9 Message Digest Generation Using SHA-512

SHA-512

- **Entrada:** mensagem $< 2^{128}$ bits
- Entrada processada em blocos de 1024 bits cada
- **Saída:** resumo de 512 bits
- Padding 10...0 é adicionado para garantir comprimento específico
- Bloco de 128 bits contém o tamanho da mensagem original (antes do padding)

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

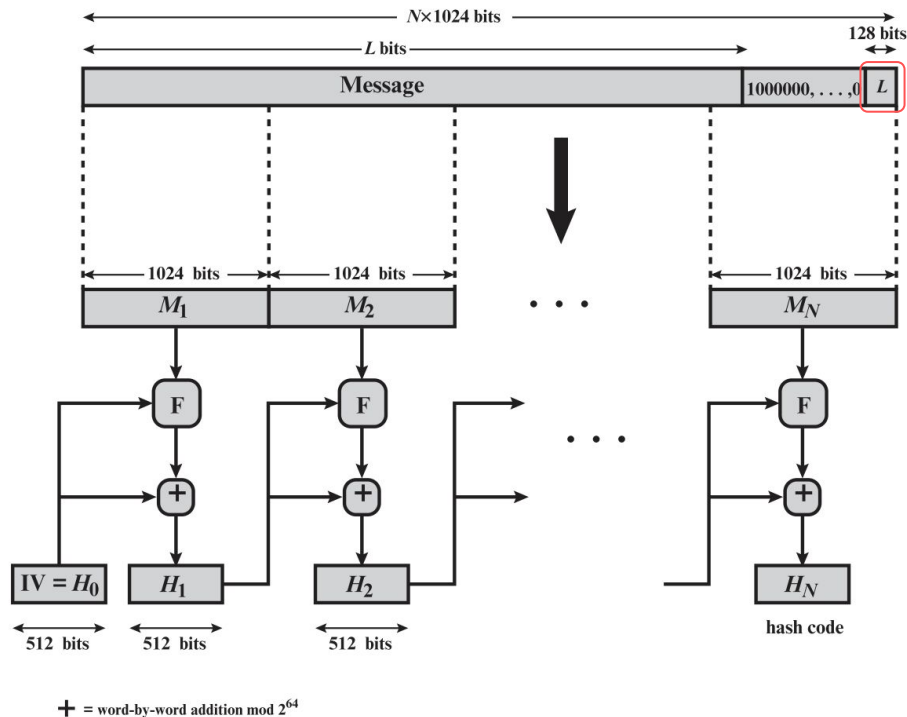


Figure 11.9 Message Digest Generation Using SHA-512

SHA-512

- **Entrada:** mensagem $< 2^{128}$ bits
- Entrada processada em blocos de 1024 bits cada
- **Saída:** resumo de 512 bits
- Padding 10...0 é adicionado para garantir comprimento específico
- Bloco de 128 bits contém o tamanho da mensagem original (antes do padding)
- Inicialização do H_0 com valores pré-determinados ([FIPS PUB 180-4](#))

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

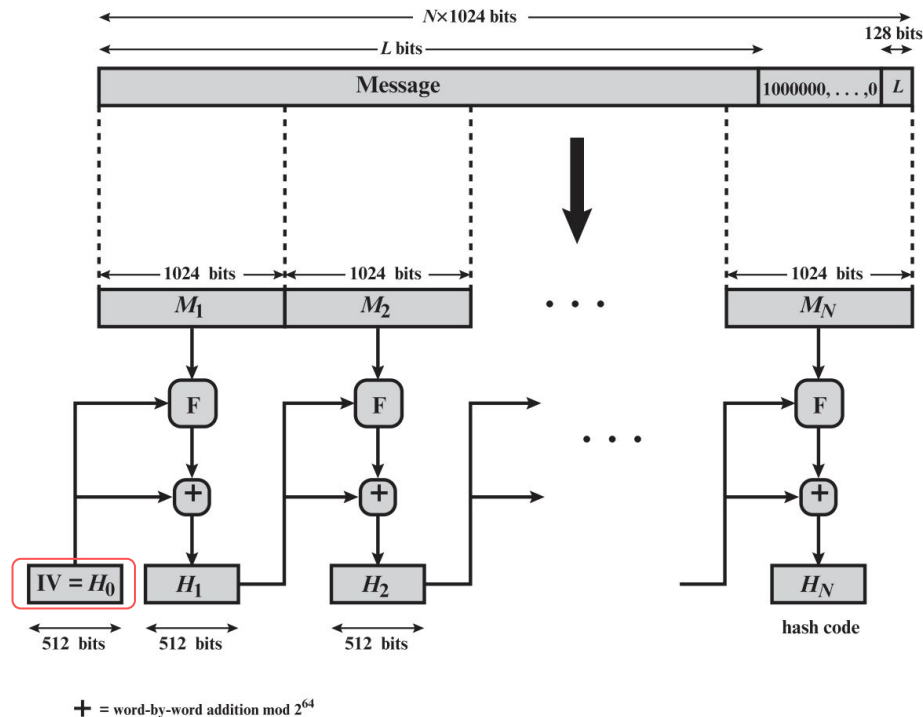


Figure 11.9 Message Digest Generation Using SHA-512

SHA-512

- **Entrada:** mensagem $< 2^{128}$ bits
- Entrada processada em blocos de 1024 bits cada
- **Saída:** resumo de 512 bits
- Padding 10...0 é adicionado para garantir comprimento específico
- Bloco de 128 bits contém o tamanho da mensagem original (antes do padding)
- Inicialização do H_0 com valores pré-determinados ([FIPS PUB 180-4](#))
- Processamento de cada bloco utilizando o valor intermediário H_{i-1}

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

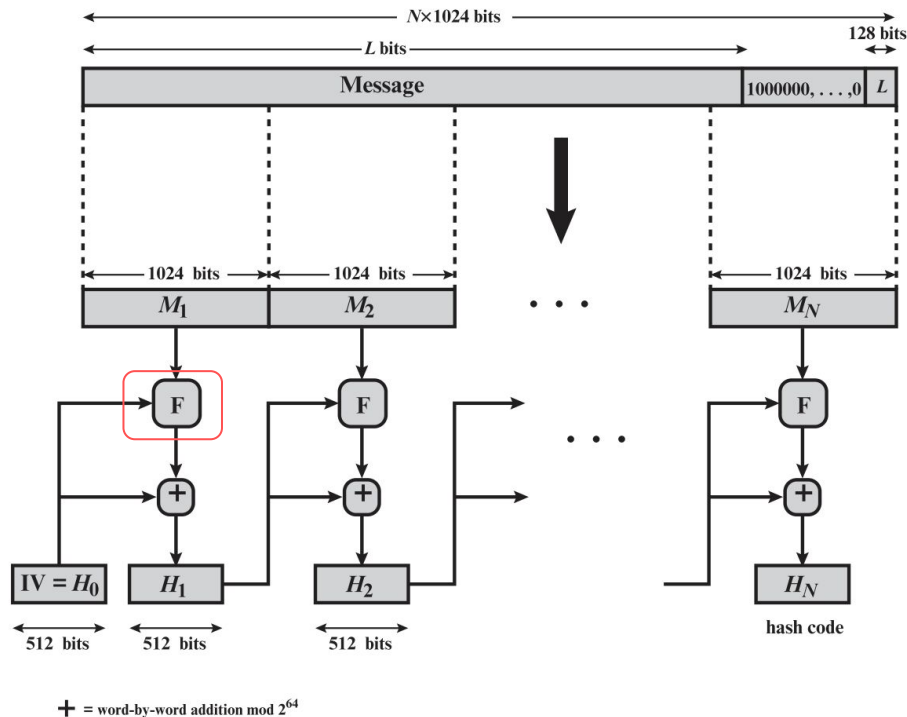


Figure 11.9 Message Digest Generation Using SHA-512

SHA-512

- A função F consistem em 80 rounds para cada bloco de texto a ser processado
- **Entrada:** bloco M_i e valor intermediário H_{i-1}
- **Saída:** valor intermediário H_i

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

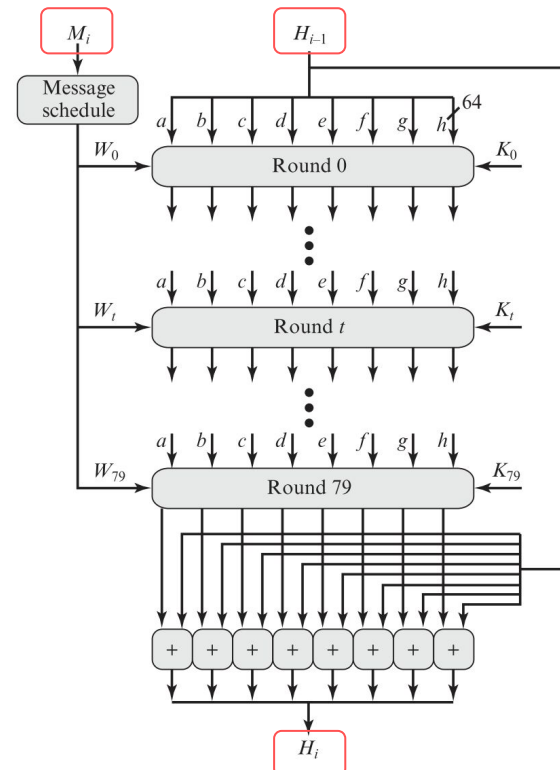


Figure 11.10 SHA-512 Processing of a Single 1024-Bit Block

SHA-512

- A função F consistem em 80 rounds para cada bloco de texto a ser processado
- **Entrada:** bloco M_i e valor intermediário H_{i-1}
- **Saída:** valor intermediário H_i
- Cada W_t consiste em 64 bits derivados da M_i
- Cada K_t é uma constante pré-determinada, que provê uma "aleatoriedade" à saída

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

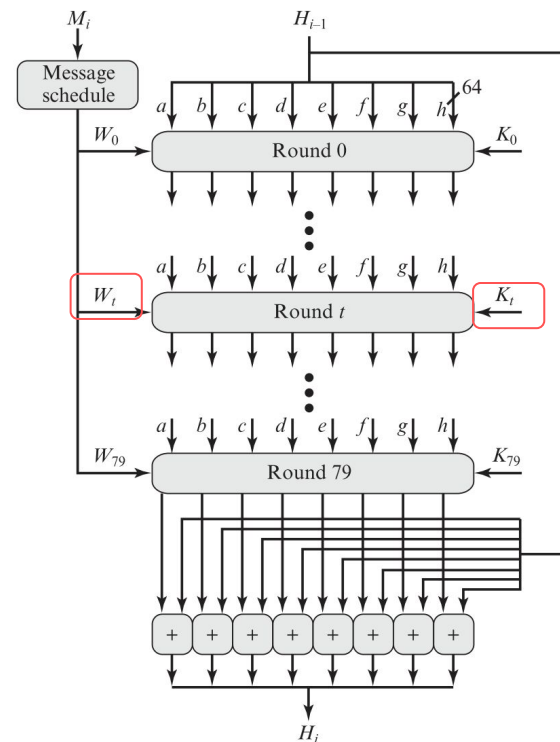


Figure 11.10 SHA-512 Processing of a Single 1024-Bit Block

SHA-512

- A função F consistem em 80 rounds para cada bloco de texto a ser processado
- **Entrada:** bloco M_i e valor intermediário H_{i-1}
- **Saída:** valor intermediário H_i
- Cada W_t consiste em 64 bits derivados da M_i
- Cada K_t é uma constante pré-determinada, que provê uma "aleatoriedade" à saída
- Cada round calcula operações elementares, como somas, operações binárias (and, xor, not) e rotações

Imagem: W. Stallings. *Cryptography and network security*. Cap 11.5

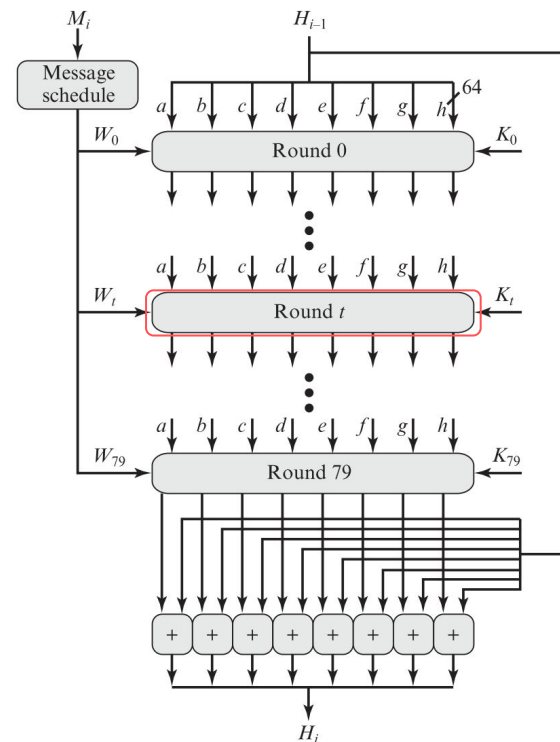


Figure 11.10 SHA-512 Processing of a Single 1024-Bit Block

SHA-512

- Os passos do algoritmo adicionam redundância e interdependência
- Cada bit da saída depende de todos os bits da entrada
- Isso faz com que seja improvável que duas mensagens aleatórias mapeiem para o mesmo hash.
- Os outros algoritmos da família SHA-2 funcionam de maneira similar
 - ver mais em [FIPS PUB 180-4](#)

Aplicações

- As funções de hash são utilizadas em uma grande variedade de aplicações e protocolos
- Exemplos de aplicações:
 - Autenticação de mensagens
 - Assinaturas digitais
 - Armazenamento seguro de senhas
 - Geração de números pseudo-aleatórios (geração de parâmetros e chaves)
 - Imutabilidade de registros em uma blockchain
 - Esquemas de comprometimento
- O uso de uma função de hash insegura compromete a aplicação

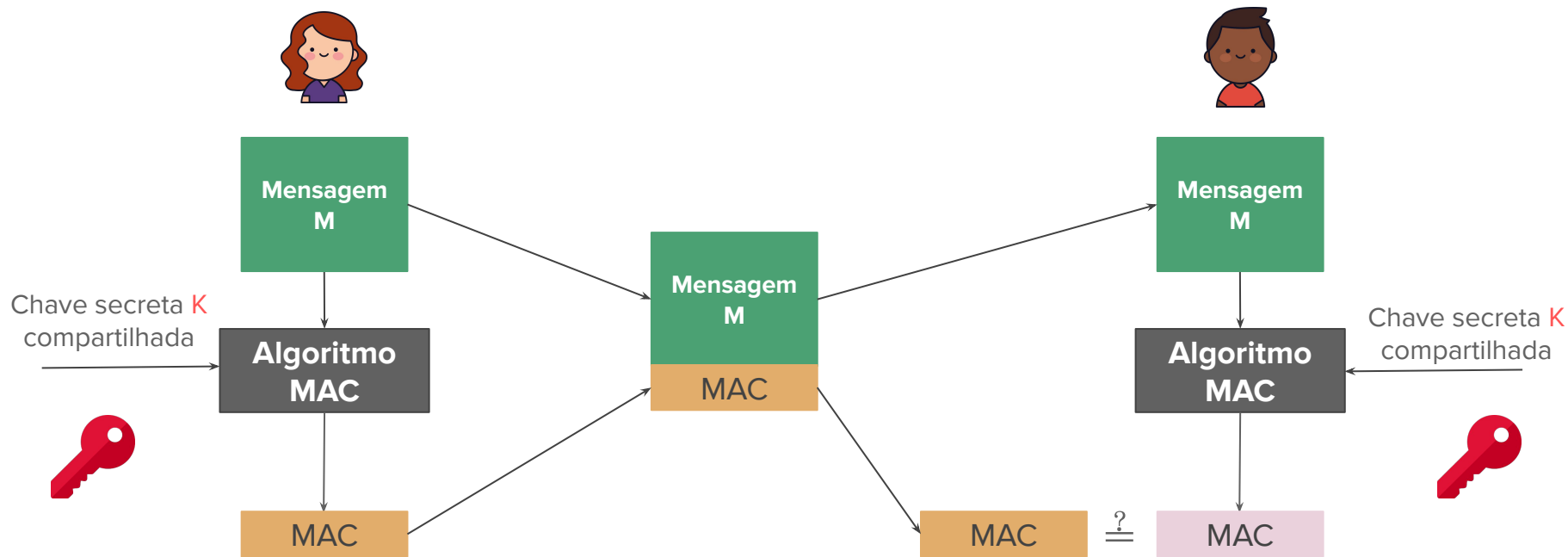
Sumário

- Definições básicas
- Funções de hash criptográficas
- **Códigos de autenticação de mensagens**
 - Definições básicas
 - Requisitos e segurança
 - HMAC
 - CMAC
- Atividades práticas

Recapitulando..

- Cifragem de mensagens provê *confidencialidade*
 - protegendo a mensagem de um atacante passivo
- Funções de hash criptográficas provêm *integridade* de dados
 - auxiliando na identificação de modificações em uma mensagem
- E a *autenticidade* dos dados?
 - garantia de que o dado é genuíno, provêm da fonte alegada e não foi alterado sem permissão
- Note que a função de hash só garante a não-alteração dos dados!

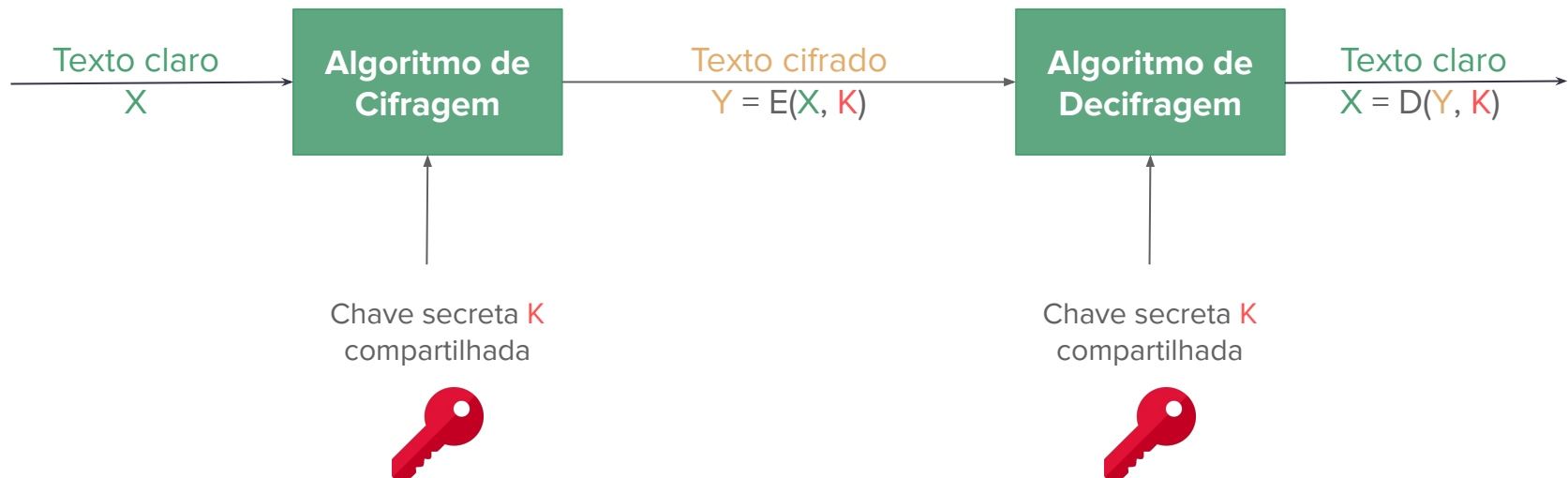
Message Authentication Code (MAC)



Message Authentication Code (MAC)

- Um MAC utiliza uma chave **K** para gerar um pequeno bloco de tamanho fixo chamado **checksum, tag** ou **MAC**
 - que é anexado à mensagem e enviado ao destinatário
- Assume-se que ambas as partes comunicantes compartilham uma chave secreta **K**
 - e apenas os dois conhecem essa chave
- Se o MAC recebido é igual ao MAC calculado pelo destinatário, ele sabe que:
 - a mensagem não foi modificada (integridade)
 - a mensagem veio da fonte esperada (autenticidade)
- Se um atacante não conhece **K**, ele não pode modificar **M** e **MAC** e passar despercebido

MAC vs Criptografia Simétrica

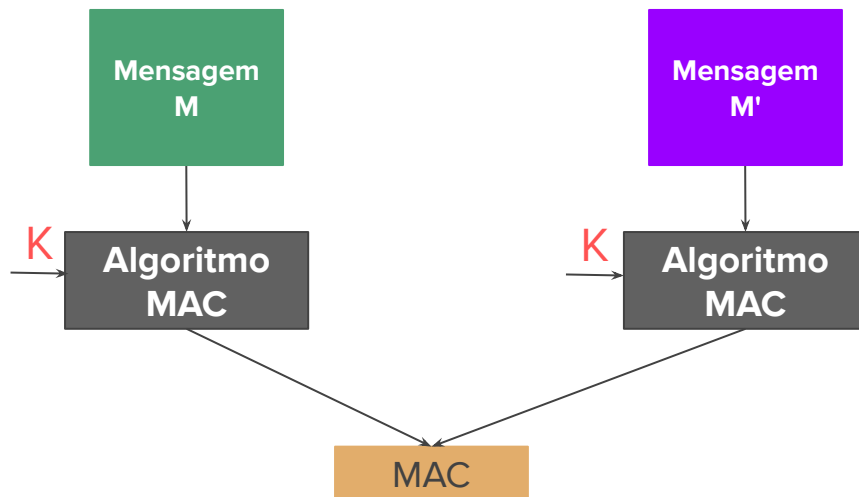


MAC vs Criptografia Simétrica

- Criptografia simétrica garante um certo nível de integridade e autenticidade
 - apenas quem possui a chave secreta pode cifrar e decifrar
 - se a mensagem for alterada durante a transmissão, a decifragem resultará em dados sem sentido
- Entretanto, MACs são mais interessantes quando:
 - a confidencialidade não é um requisito importante.
 - por exemplo, verificação de autenticidade de programas de computador
 - existe uma alta demanda e necessidade de eficiência
 - não é possível decifrar todas as mensagens recebidas, a autenticação é feita de maneira seletiva
 - a mensagem é enviada para vários destinatários (broadcast)
 - mensagens são enviadas em claro para todos, mas apenas um destinatário possui a chave secreta e verifica a integridade/autenticidade
 - se ocorrer um problema, os outros destinatários são notificados por um alarme geral

Requisitos e segurança

- O **MAC** é uma função de muitos-para-um, ou seja, para uma chave **K**, muitas mensagens diferentes produzem o mesmo **MAC**
 - questões de colisão também são observadas aqui



Requisitos e segurança

- Assumimos que um atacante conheça a mensagem **M** e o **MAC** produzido, mas não conheça **K**
- O algoritmo MAC deve satisfazer os seguintes requisitos:
 - o atacante não deve ser capaz de criar uma outra mensagem **M'** que tenha o mesmo **MAC** da mensagem conhecida **M**
 - o algoritmo deve ser uniformemente distribuído, de maneira que um ataque de força bruta seja muito difícil
 - deve ser muito difícil para um atacante modificar **M** de maneira a produzir uma mensagem modificada **M'** que tenha o mesmo **MAC** da primeira.
- Medimos a segurança de um algoritmo MAC comparando o esforço para quebrá-lo com o esforço requerido para um ataque de força bruta
 - um algoritmo ideal requer um esforço igual ou superior

Algoritmos MAC

- Existem diversas maneiras de implementar um algoritmo de MAC
- Vamos explorar duas possibilidades:
 - MACs baseados em funções de hash (HMAC)
 - MACs baseados em cifras simétricas (CMAC)

Keyed-Hash Message Authentication Code (HMAC)

- Funções de hash não possuem chaves e portanto não podem ser utilizadas diretamente na construção de MACs
- O HMAC surgiu da proposta de incorporação de uma chave secreta em uma função de hash
- Os principais objetivos do HMAC são:
 - utilizar funções de hash disponíveis sem modificá-las
 - permitir fácil substituição da função de hash, caso uma mais rápida ou mais segura seja criada
 - preservar a rápida performance das funções de hash
 - usar e manipular chaves de maneira simplificada
 - análise criptográfica bem compreendida, baseada na função de hash
- Padronização do HMAC: [FIPS PUB 198-1](#)
- Descrição e detalhes: [RFC 2104](#)

HMAC

- $\text{HMAC}(K, M) = H[(K \oplus \text{opad}) \parallel H[(K \oplus \text{ipad}) \parallel M]]$

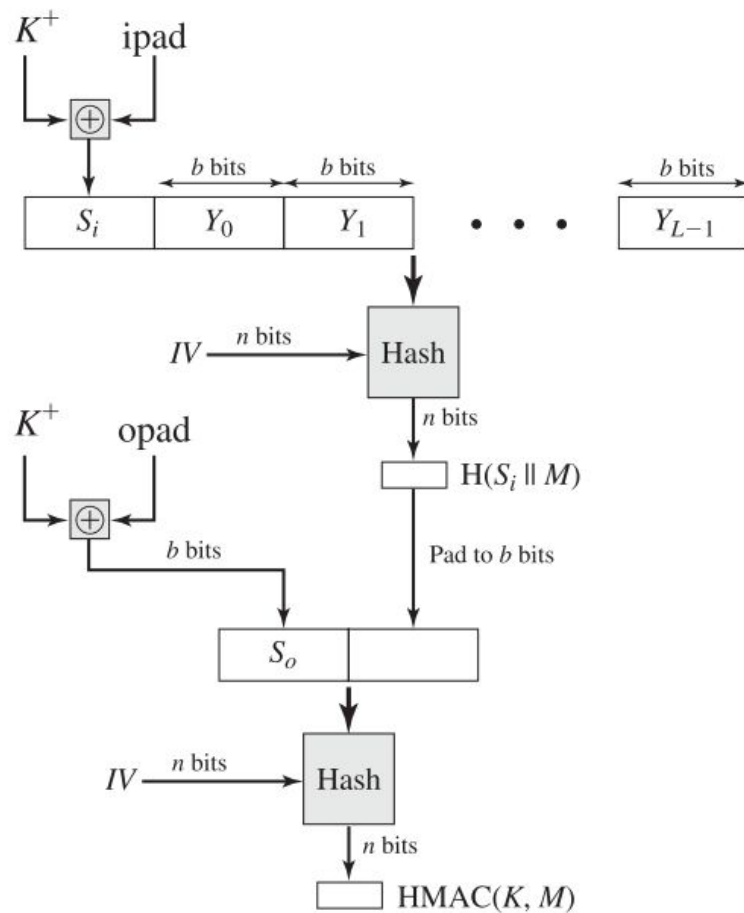


Figure 12.5 HMAC Structure

Fonte: W. Stallings. Cryptography and network security. Cap 12.5

HMAC

- $\text{HMAC}(K, M) = H[(K \oplus \text{opad}) \parallel H[(K \oplus \text{ipad}) \parallel M]]$
- Mensagem M é dividida em L blocos de b bits cada

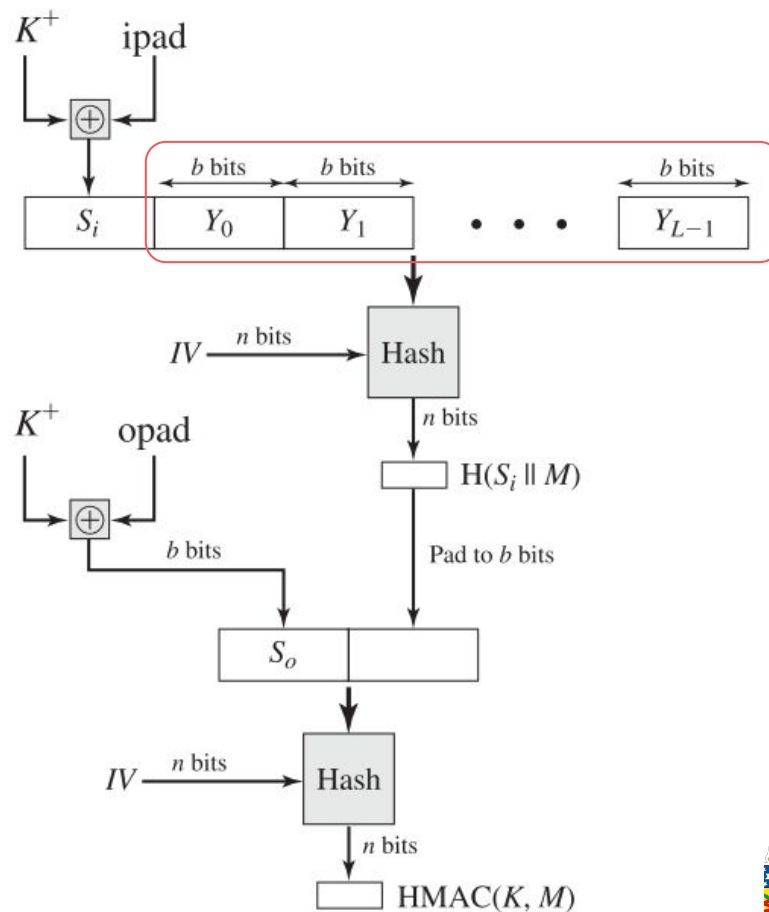


Figure 12.5 HMAC Structure

Fonte: W. Stallings. Cryptography and network security. Cap 12.5

HMAC

- $\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$
- Mensagem M é dividida em L blocos de b bits cada
- K^+ é a chave K concatenada com 0s à esquerda para completar b bits

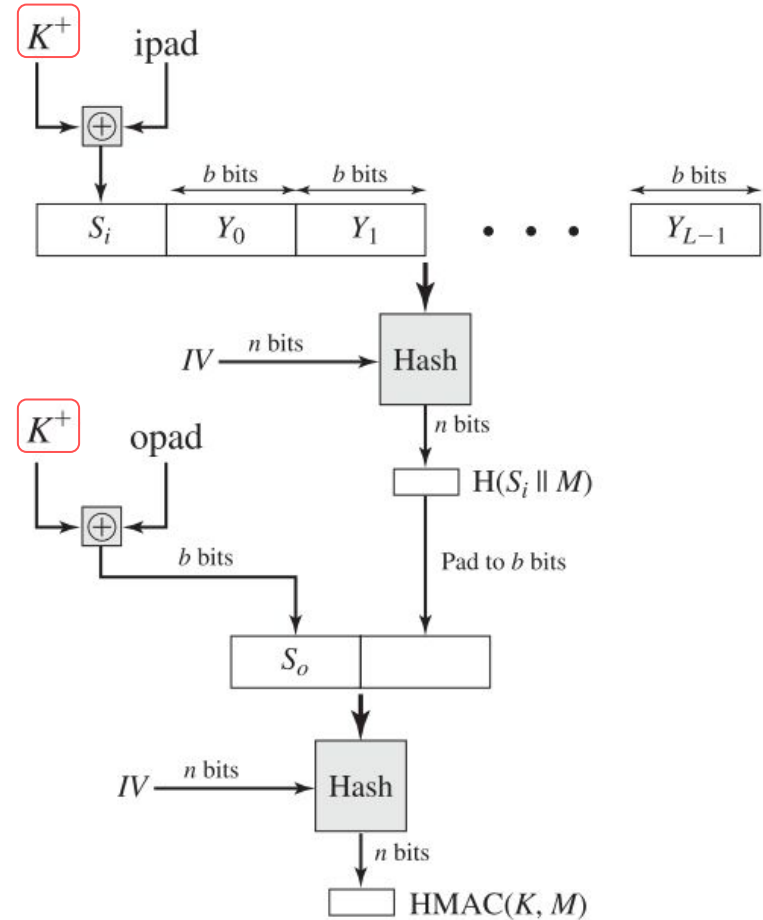


Figure 12.5 HMAC Structure

Fonte: W. Stallings. Cryptography and network security. Cap 12.5

HMAC

- $\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$
- Mensagem M é dividida em L blocos de b bits cada
- K^+ é a chave K concatenada com 0s à esquerda para completar b bits
- ipad : 00110110 = 0x36 (repetido $b/8$ vezes)
- opad : 01011100 = 0x5C (repetido $b/8$ vezes)

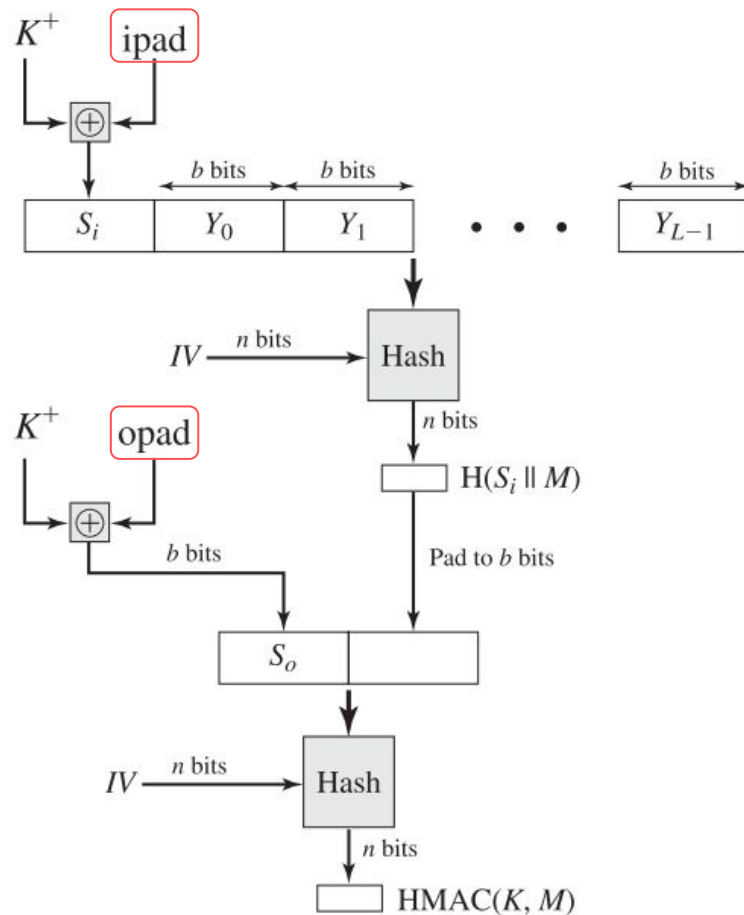


Figure 12.5 HMAC Structure

Fonte: W. Stallings. Cryptography and network security. Cap 12.5

HMAC

- $\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$
- Mensagem M é dividida em L blocos de b bits cada
- K^+ é a chave K concatenada com 0s à esquerda para completar b bits
- ipad : 00110110 = 0x36 (repetido $b/8$ vezes)
- opad : 01011100 = 0x5C (repetido $b/8$ vezes)
- opad e ipad fazem um flip dos bits de K

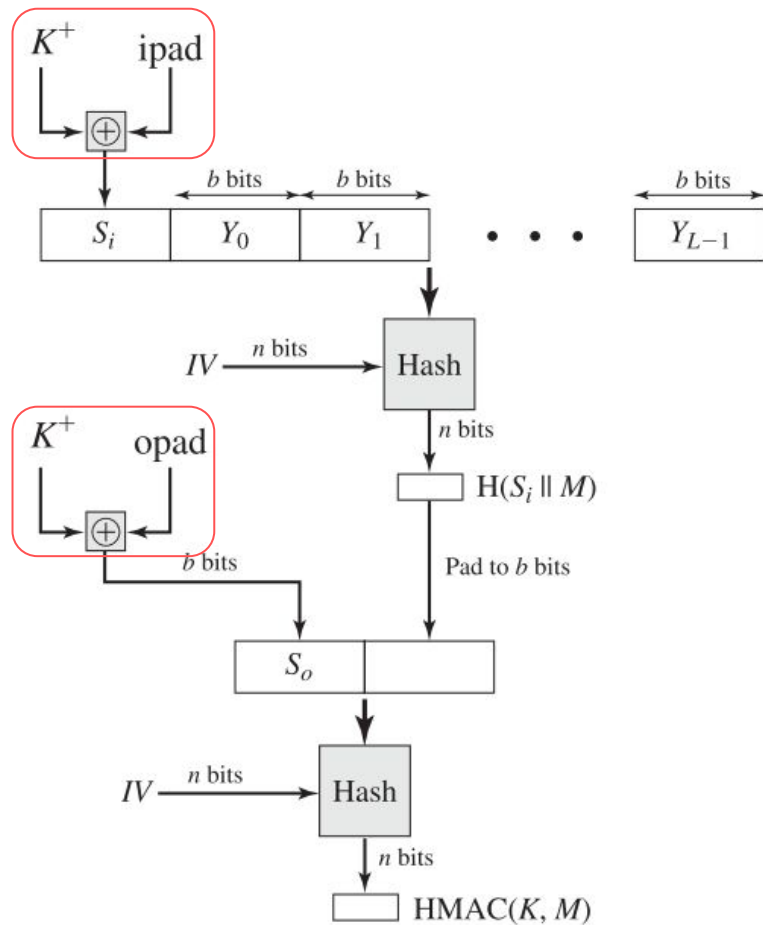


Figure 12.5 HMAC Structure

Fonte: W. Stallings. Cryptography and network security. Cap 12.5

HMAC

- $\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$
- Mensagem M é dividida em L blocos de b bits cada
- K^+ é a chave K concatenada com 0s à esquerda para completar b bits
- ipad: 00110110 = 0x36 (repetido $b/8$ vezes)
- opad: 01011100 = 0x5C (repetido $b/8$ vezes)
- opad e ipad fazem um flip dos bits de K
- podemos pré-computar o cálculo de S_i e S_o

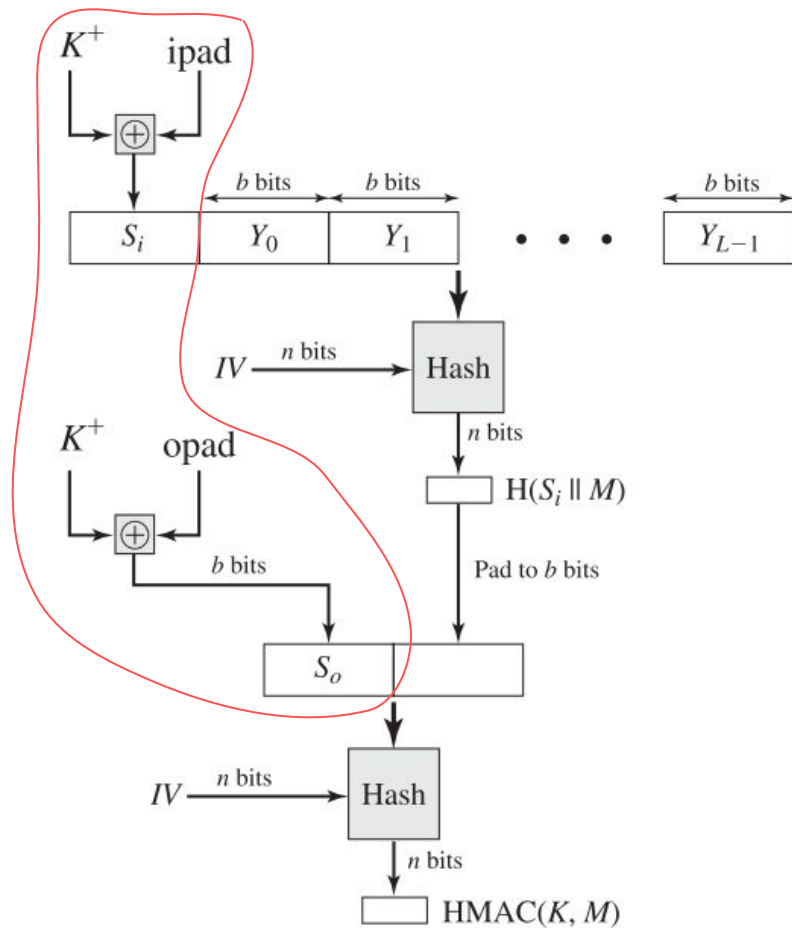


Figure 12.5 HMAC Structure

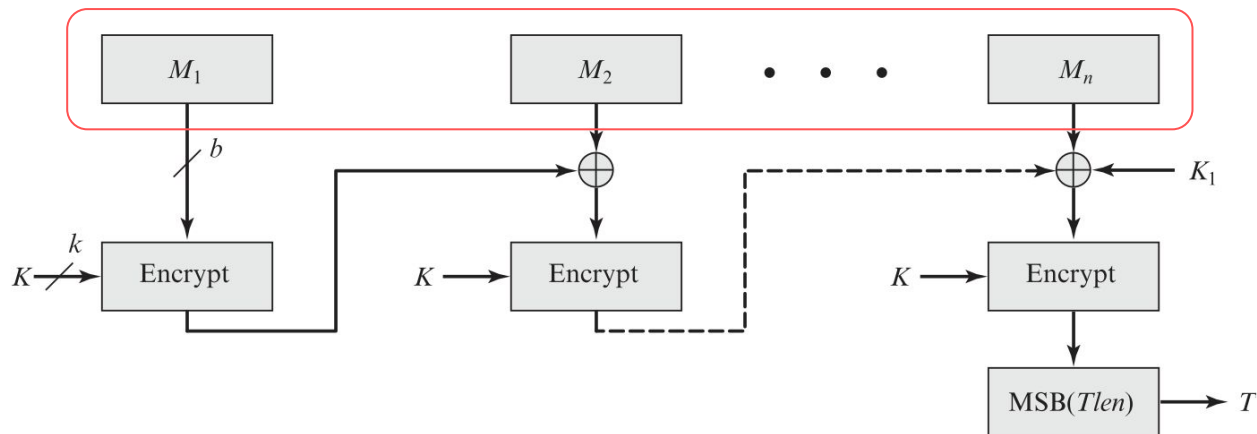
Fonte: W. Stallings. Cryptography and network security. Cap 12.5

Segurança do HMAC

- A segurança de qualquer MAC baseado em funções de hash depende da segurança da função de hash utilizada;
- A probabilidade de sucesso de um ataque no HMAC é equivalente à probabilidade de se encontrar colisões na função de hash utilizada;

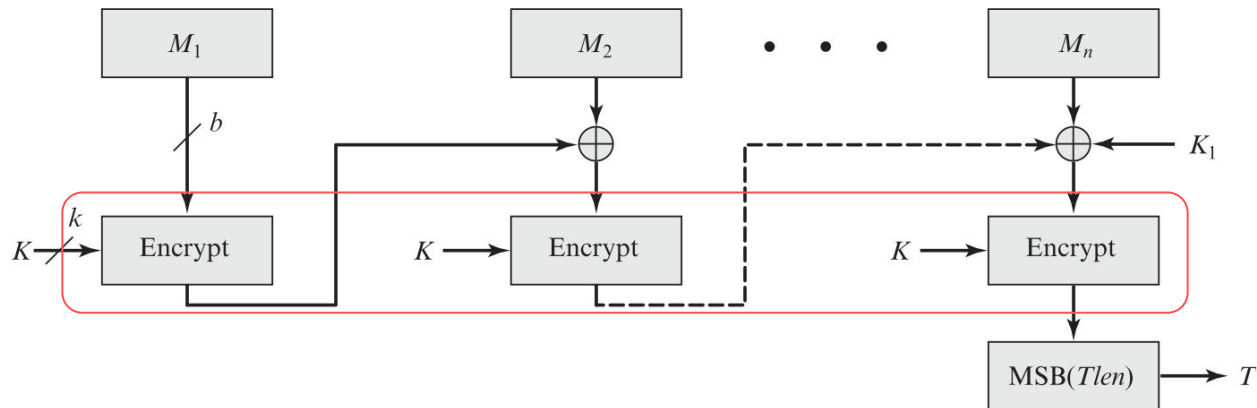
Cipher-Based Message Authentication Code (CMAC)

- Mensagem é dividida em n blocos de tamanho b



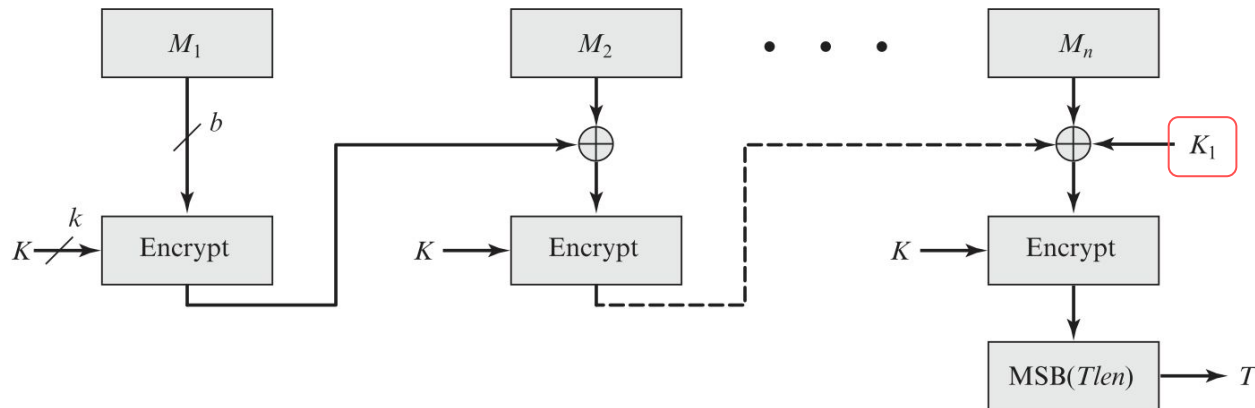
Cipher-Based Message Authentication Code (CMAC)

- Mensagem é dividida em n blocos de tamanho b
- Cada bloco é cifrado utilizando um algoritmo simétrico e uma chave K
 - b depende do algoritmo escolhido



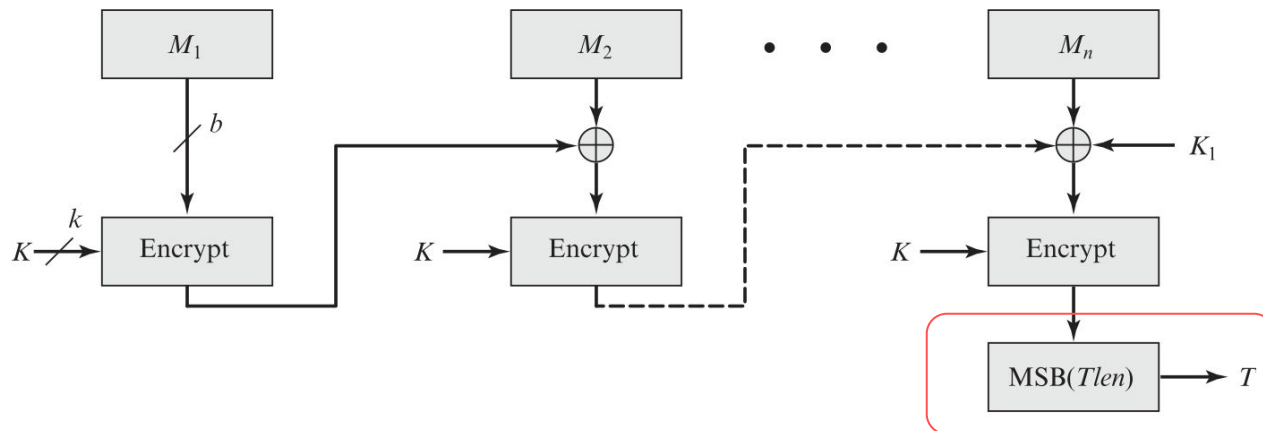
Cipher-Based Message Authentication Code (CMAC)

- Mensagem é dividida em n blocos de tamanho b
- Cada bloco é cifrado utilizando um algoritmo simétrico e uma chave K
 - b depende do algoritmo escolhido
- Uma sub-chave K_1 é utilizada no último bloco



Cipher-Based Message Authentication Code (CMAC)

- Mensagem é dividida em n blocos de tamanho b
- Cada bloco é cifrado utilizando um algoritmo simétrico e uma chave K
 - b depende do algoritmo escolhido
- Uma sub-chave K_1 é utilizada no último bloco
- A saída consiste nos bits mais significativos (mais à esquerda)



Considerações finais

- CMAC é especificado na [NIST SP 800-38B](#)
- A cifra simétrica indicada é AES ($b = 126$)
 - o 3DES ($b = 64$) não é mais permitido desde 2023
- Funções de hash geralmente são mais eficientes em software do que cifras simétricas
- Aplicações com hardware dedicado para cifras podem oferecer um desempenho superior ao CMAC
- HMAC oferece maior flexibilidade e implementação mais simples, já que pode ser utilizado com diversas funções de hash que são amplamente disponíveis nas bibliotecas

Aplicações

- No protocolo TLS (versão 1.2), utilizado para garantir a integridade e autenticidade das mensagens trocadas;
- No JSON Web Token (JWT) para prover segurança nos tokens;
- No Amazon Web Services (AWS), para garantir integridade e autenticidade de mensagens trocadas entre os diferentes componentes e serviços e autenticar requisições de usuários;
- entre outras.

Resumo

- Funções de hash transformam dados de qualquer tamanho em "resumos" de tamanhos fixos;
- Utilizados na garantia de integridade de dados;
- Para serem seguros, as funções de hash criptográficas precisam seguir alguns requisitos;
 - caminho único, difícil encontrar colisões, efeito avalanche, saída com aparência aleatória
- Os principais algoritmos são os SHA, que processam a entrada em blocos e possuem diversos rounds de operações
- Escolha algoritmos seguros e mantenha-se atualizado sobre novas vulnerabilidades!

Resumo

- Message Authentication Codes (MACs) são utilizados na garantia de integridade e autenticidade de dados;
- Produzem uma saída de tamanho fixo conhecida por MAC, *tag* ou *checksum*;
- Podem ser baseadas em funções de hash ou em cifras simétricas;
- A segurança dos MACs depende da segurança da função de hash ou cifra utilizada;

Sumário

- Definições básicas
- Funções de hash criptográficas
- Códigos de autenticação de mensagens
- **Atividades práticas**

Atividade: calculando hashes

- Vamos praticar utilizando o openssl:

openssl dgst -algoritmo arquivo

- Calcule o hash do arquivo qualquer e faça o seguinte experimento:
 - utilize as funções SHA-256 e SHA-512 e observe a diferença nas saídas
- Modifique o arquivo e gere novamente o hash
 - observe o efeito avalanche ao se modificar o valor de entrada
- Calcule o hash das imagens **ship.jpg** e **plane.jpg** disponíveis no moodle
 - utilize a função md5, o que acontece? E se utilizar o sha256?

Atividade: calculando MACs

- Vamos praticar utilizando o openssl:

```
openssl dgst -hmac "chave" -sha256 nome-arquivo
```

- Calcule o HMAC de um arquivo qualquer
- Modifique o arquivo e gere novamente o HMAC
 - observe o efeito avalanche ao se modificar o valor de entrada
- Modifique a chave e observe a modificação no valor de saída
- Modifique a função de hash e observe a modificação no tamanho da saída

Referências

- W. Stallings. *Cryptography and network security*. 7a edição.
 - Hash: Capítulos 11.1, 11.3, 11.5, 11.6
 - Autenticação: Capítulos 12.1, 12.2, 12.3, 12.4, 12.5, 12.5
- D. Stinson e M. Paterson. *Cryptography: Theory and Practice*. 4a edição.
 - Hash: Capítulos 5.1, 5.2
 - Autenticação: Capítulos 5.5 e 5.6
- [RFC 2104](#)
- [FIPS PUB 198-1](#)
- [NIST SP 800-38B](#)
- imagens: Flaticon.com